ShopEZ E-Commerce Platform

Full Stack Development (MERN)ProjectDocumentation

1. Introduction

- **Project Title:** ShopEZ E-Commerce Platform
- **Team Members and Roles:**
    - *Frontend Developer:* [Enter Name] - Built responsive UI layout components, page routing, and context APIs.
    - *Backend Developer:* [Enter Name] - Set up Express server routes, controllers, middleware, and database models.
    - *Database Administrator:* [Enter Name] - Modeled Mongoose database schemas, indexing, and seed scripts.
    - *UI/UX Designer:* [Enter Name] - Structured the light/dark themes, CSS layout rules, and interactive elements.

2. Project Overview

- **Purpose:** ShopEZ is a robust, full-featured MERN (MongoDB, Express.js, React, Node.js) stack e-commerce platform designed to deliver a smooth and secure online shopping experience. It connects a React-based frontend with a modular Node.js API to process product catalog browsing, cart operations, user/admin authentication, and transactional checkouts.
- **Core Features:**
    - **Secure Authentication:** Dual user-authentication structures supporting role-based logins (Customer vs. Admin) backed by JWT (JSON Web Tokens) and bcrypt password hashing.
    - **Interactive Product Catalog:** Dynamic grid sorting and filtering by categories (Mobiles, Electronics, Sports-Equipment, Fashion, Groceries), gender (Men, Women, Unisex), text searches, and pricing metrics (Popular, priceLowHigh, priceHighLow, Discount).
    - **Advanced Details View:** Dedicated product pages featuring visual image carousels, responsive item attributes (size selection or storage capacity), live pricing computations, and reviews section.

- **Persistent Shopping Cart:** Fully synchronized database-driven shopping cart allowing item additions, quantity manipulation, and item removal linked to user sessions.
- **Checkout & Invoice System:** Real-time billing calculations including MRP, discounts, shipping fee exemption (free delivery for orders > ₹999), and standard 18% GST calculation. Supports UPI, Card, and Cash on Delivery payment options.
- **User Profile & Order History:** Interactive user area showing current registration details and a detailed order history log, complete with active status trackers and cancellation triggers.
- **Admin Dashboard & Inventory Control:** Central management suite exposing high-level sales statistics (Total Revenue, Active Users, Total Products, Total Orders), live inventory tables (Create, Read, Update, Delete products), order execution tracking (Updating status from "order placed" through "delivered"), and user account promotions/deletions.

3. Architecture

The application implements a decoupled client-server architecture with the database acting as a single source of truth:

mermaid

graph TD

Client[React Frontend - Vite] <-->|HTTP REST APIs + JWT Bearer| Server[Node.js / Express API Server]

Server <-->|Mongoose ODM| DB[(MongoDB Database)]

- **Frontend (Client):** Built using **React 19** and **Vite** for fast HMR (Hot Module Replacement). State management is unified under a custom React Context provider (

ShopContext.jsx) handling network API fetches, search states, and user sessions. Styled using Vanilla CSS variables supporting responsive designs and custom light/dark color palettes.

- **Backend (Server):** Powered by **Node.js** and **Express.js**. Configured with standard security middleware including cors (restricted to frontend origins) and express.json(). Implements a controller-router pattern to divide logic, with custom token verification middleware (

authMiddleware.js) and error handlers (

errorMiddleware.js).

- **Database (MongoDB):** Utilizes MongoDB utilizing Mongoose ODM schemas. The models comprise:
    1. **User**
       Model: Contains username, email, password, usertype (Admin/Custom er), and pre-save hooks to sync role metrics.
    2. **Product Model:** Stores product metadata
       including title, description, mainImg, carousel, sizes, category, gender, price, and discount.
    3. **Admin Model:** Separate collection for dedicated admin accounts containing elevated dashboard permissions.
    4. **Cart Model:** Stores flat database records mapping users to their selected items, prices, sizes, and quantities.
    5. **Order Model:** Stores permanent invoice snapshots containing shipping addresses, item quantities, delivery timetables, and order tracking statuses.

4. Setup Instructions

Prerequisites

- Node.js (v18.0.0 or higher recommended)
- npm (bundled with Node.js)
- MongoDB Community Server (running locally on port 27017) or a MongoDB Atlas account.

Installation Steps

1. **Clone the Repository**

bash

git clone <repository-url>

cd shopez-ecommerce

2. **Install Client Dependencies**

bash

cd Client

npm install

3. **Install Server Dependencies**

bash

cd ../Server

npm install

4. **Configure Environment Variables** Create a new file named .env in the root of the Server directory and populate it with:

env

PORT=8000

MONGODB_URI=mongodb://127.0.0.1:27017/shopez

MONGO_URI=mongodb://127.0.0.1:27017/shopez

JWT_SECRET=shopez_super_secret_jwt_key_2024

JWT_EXPIRES_IN=7d

NODE_ENV=development

5. **Start Database Service** Ensure your local MongoDB daemon is running:
    - *Windows (PowerShell):* Start-Service MongoDB
    - *macOS/Linux:* brew services start mongodb-community

5. Folder Structure

ShopEZ E-Commerce Application/

├── Client/ # Frontend Application (React + Vite)

│ ├── public/ # Static asset folder

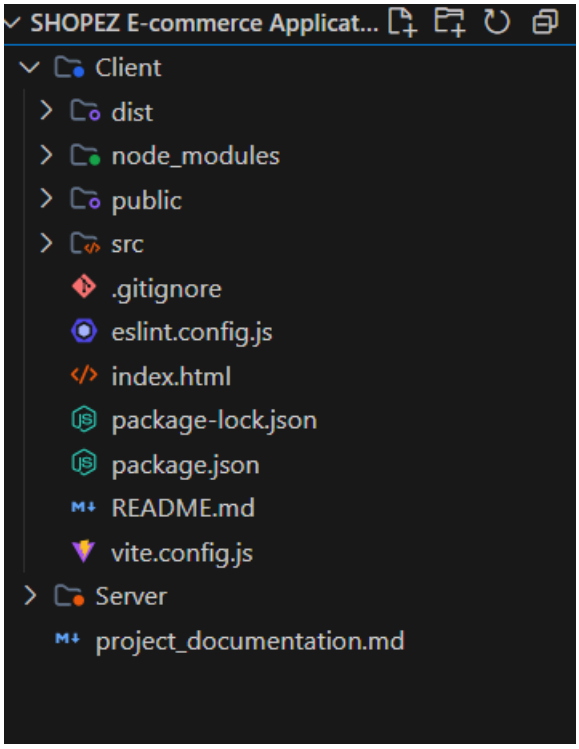
│ ├── src/ # React source files

│ │ ├── assets/ # Icons and general styling assets

│ │ ├── components/ # Reusable interface widgets

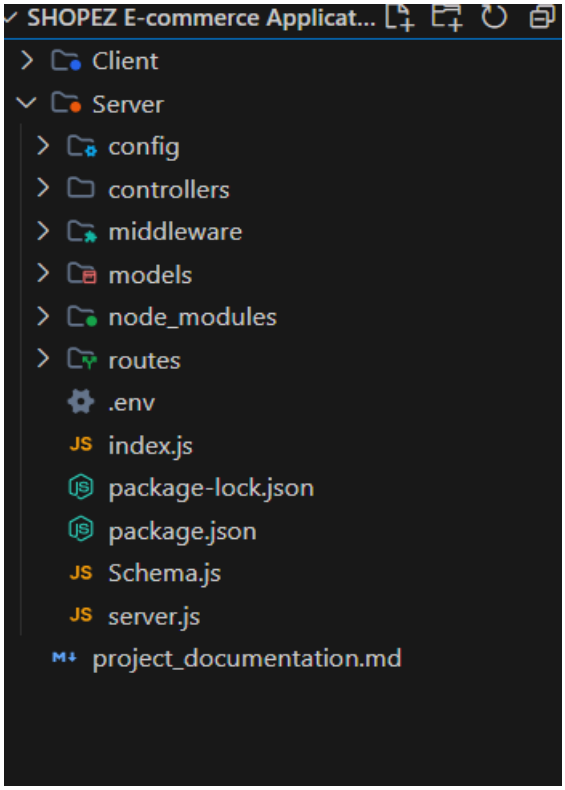
│ │ │ ├── Header.jsx # Main navigation and search bar

```
│   │   │   ├── Footer.jsx       # Info footer
│   │   │   └── ProductCard.jsx  # Reusable product grid cards
│   │   ├── context/             # Global State Managers
│   │   │   └── ShopContext.jsx  # Core app API context engine
│   │   ├── pages/               # Routing View Screens
│   │   │   ├── LandingPage.jsx  # Promo homepage
│   │   │   ├── CatalogPage.jsx  # Filterable product grid
│   │   │   ├── ProductDetailsPage.jsx # Detailed specs & buy buttons
│   │   │   ├── CartPage.jsx     # Cart item edits
│   │   │   ├── CheckoutPage.jsx # Shipping & payment configuration
│   │   │   ├── OrderConfirmationPage.jsx # Post-purchase invoice receipt
│   │   │   ├── ProfilePage.jsx  # Transaction log viewer
│   │   │   ├── AuthPage.jsx     # Client registration / login portal
│   │   │   └── AdminPage.jsx    # Inventory & status dashboard
│   │   ├── App.css              # Glassmorphic themes & responsive rules
│   │   ├── App.jsx              # Routing and site layout container
│   │   ├── index.css            # Base resets and custom styles
│   │   └── main.jsx             # React entry mounting script
│   ├── package.json             # React dependencies & build scripts
│   └── vite.config.js           # Vite server settings
│
```




```
└── Server/                 # Backend REST API Server (Node + Express)
    ├── config/             # Server configuration directories
    │   └── db.js           # MongoDB Mongoose connector
    ├── controllers/        # Express Route Handlers (Business Logic)
    │   ├── adminController.js   # Admin registrations, analytics, user controls
    │   ├── cartController.js    # Database cart CRUD handlers
    │   ├── orderController.js   # Checkout processing, status changes, cancellations
    │   ├── productController.js # Product catalogs, database seeding, Admin inventory CRUD
    │   └── userController.js    # JWT user registration and profiling
    ├── middleware/         # Security and filter utilities
    │   ├── authMiddleware.js    # JWT authorization and Admin access guards
    │   └── errorMiddleware.js   # Central error catcher
    ├── models/             # Mongoose MongoDB Data Schemas
    │   ├── adminModel.js        # Admin credential schemas
```

```
│   ├── cartModel.js        # Active cart items schema
│   ├── orderModel.js       # Invoice transaction schema
│   ├── productModel.js     # Catalog items schema
│   └── userModel.js        # Customer profiles schema
├── routes/                 # Express Router Endpoints
│   ├── adminRoutes.js      # Admin actions endpoints
│   ├── cartRoutes.js       # Cart CRUD endpoints
│   ├── orderRoutes.js      # Transactions endpoints
│   ├── productRoutes.js    # Catalog CRUD endpoints
│   └── userRoutes.js       # User profile endpoints
├── .env                    # Server environment configurations
├── index.js                # App entry script (Loads Dotenv)
├── server.js               # App bootstrap (Express setup, CORS, routing mount)
└── package.json            # Backend dependencies & script configuration
```



6. Running the Application

To run the application locally in development mode, open two separate terminal instances:

Terminal 1: Backend Server

bash

cd Server

npm run dev

- Launches Express API server on **http://localhost:8000** (using nodemon).

Terminal 2: Frontend Client

bash

cd Client

npm run dev

- Launches React development server on **http://localhost:5173**.

7. API Documentation

All API endpoints are prefixed with /api and expect JSON payloads where appropriate.

1. User Authentication (/api/users)

- POST /register - Register a new customer. Payload: { username, email, password, usertype }
- POST /login - Log in a customer. Returns JWT token and user profile object. Payload: { email, password }
- POST /logout - Logs user out (Client handles session deletion).
- GET /profile - Fetches authenticated user info. *[Headers: Authorization Bearer]*
- PUT /profile - Modifies user metadata (username, email, password). *[Headers: Authorization Bearer]*

2. Products Catalog (/api/products)

- GET / - Returns product array. Supports query parameters:
    - category: Comma-separated list (e.g. mobiles,Electronics).
    - gender: Comma-separated list (e.g. Men,Unisex).
    - search: String queries matched against title or description.

        - sortBy: Options include Popular, priceLowHigh, priceHighLow, or Discount.
        - *Note: Automatically seeds a pre-defined 32-item catalog if the database contains fewer than 30 items.*
    - GET /:id - Returns product object matching target ID.
    - POST / - Creates a new product. *[Headers: Authorization Bearer (Admin-only)]*
    - PUT /:id - Updates an existing product details. *[Headers: Authorization Bearer (Admin-only)]*
    - DELETE /:id - Deletes product from inventory. *[Headers: Authorization Bearer (Admin-only)]*
    - POST /:id/reviews - Stubbed endpoint for reviewing products. *[Headers: Authorization Bearer]*

3. Shopping Cart (/api/cart)
    - GET /?userId=<id> - Returns active cart item documents formatted inside { product, quantity, size } blocks.
    - POST / - Adds/updates product item. Payload: { userId, productId, quantity, size }
    - DELETE /?userId=<id>&productId=<id> - Removes a product or specific item database ID.
    - DELETE /clear?userId=<id> - Empties the user's cart.

4. Order Processing (/api/orders)
    - POST / - Submits a checkout list. Clears active user cart if successful. Payload: { userId, items, shippingAddress, paymentMethod, specificRequirements }
    - GET /?userId=<id> - Fetches past orders. Automatically attaches subtotal, 18% GST tax, shipping cost, and final pricing.
    - GET /:id - Fetches single order invoice records.
    - DELETE /:id - Cancels an active order (changes status to cancelled).
    - GET /admin/all - List all orders placed across the system. *[Headers: Authorization Bearer (Admin-only)]*
    - PUT /:id/status - Updates order tracking status. Payload: { status }. *[Headers: Authorization Bearer (Admin-only)]*

5. Admin Utilities (/api/admin)

- POST /register - Register a new administrative account. Payload: { name, email, password }
- POST /login - Log in an admin account. Returns admin JWT token. Payload: { email, password }
- GET /dashboard - Fetches high-level database stats (active accounts, product counts, total orders, sales revenue) and the 5 most recent orders. *[Headers: Authorization Bearer (Admin-only)]*
- GET /users - Returns a list of all user profiles. *[Headers: Authorization Bearer (Admin-only)]*
- PUT /users/:id/promote - Promotes user to administrative status. *[Headers: Authorization Bearer (Admin-only)]*
- DELETE /users/:id - Deletes target user account. *[Headers: Authorization Bearer (Admin-only)]*

8. Authentication & Authorization

ShopEZ utilizes a JWT Bearer Token protocol to protect user resources and implement role-based controls:

[Client Login] ⟶ Hashed Verify (bcrypt) ⟶ Generate JWT (jsonwebtoken) ⟶ [Saved in LocalStorage]

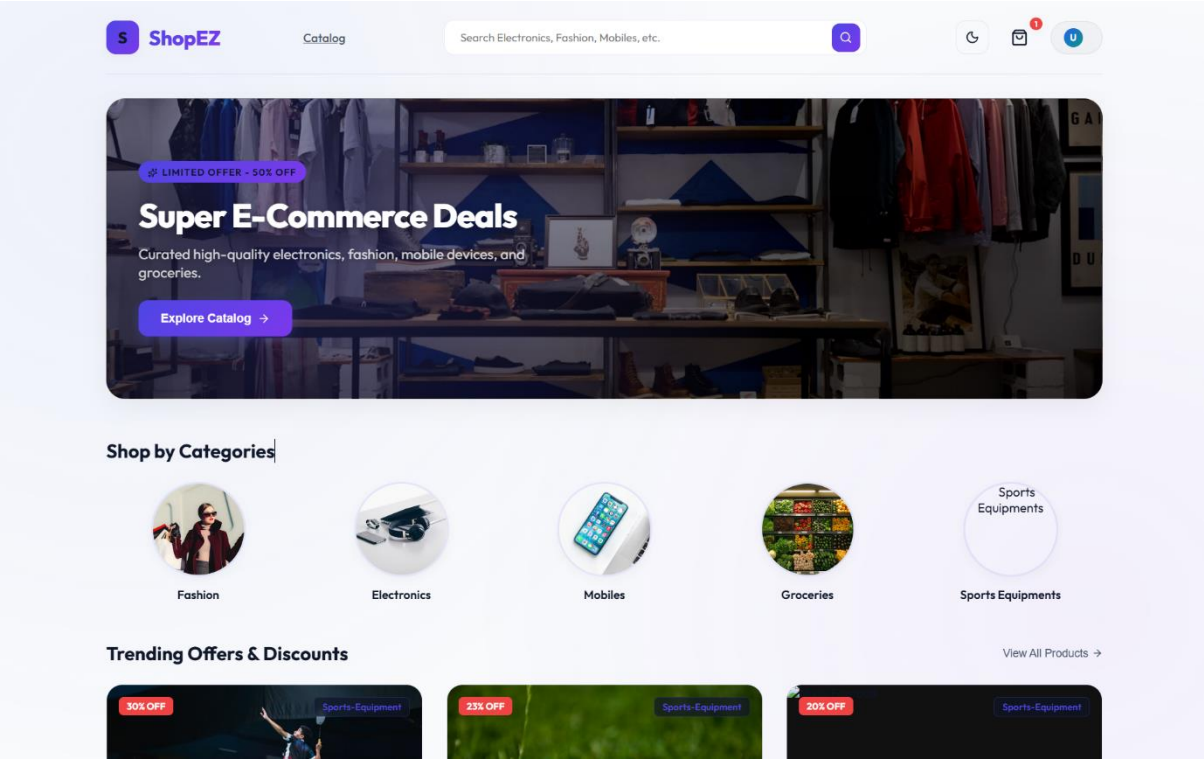
[Client Request] ⟶ Headers: Authorization: Bearer <Token> ⟶ Express authMiddleware ⟶ Access Granted

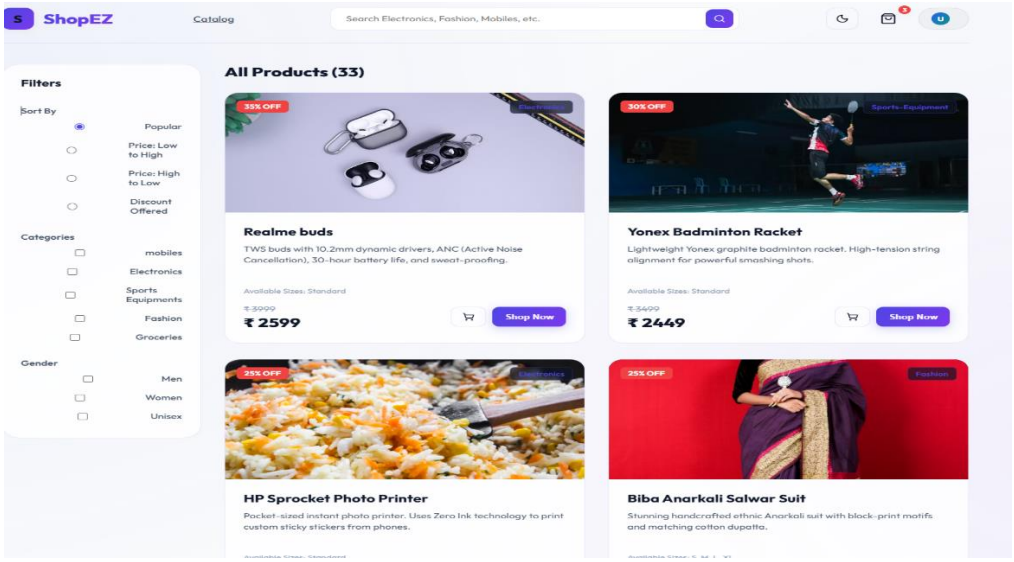
- **Token Generation:** Upon successful login or registration, the backend signs a JSON Web Token payload containing the user's Mongoose ID (id), role, and isAdmin flag, using the secret string stored in JWT_SECRET.
- **Request Verification:** Protected routes on both frontend and backend use this token:
    - **Backend:** The protect middleware checks for the presence of the Bearer token inside the request's Authorization header, decrypts it using jwt.verify(), and attaches the resulting Mongoose user record (req.user) to the request object.
    - **Frontend:** Intercepts protected page links (e.g. Profile, Checkout, Admin area) and attaches the token found in localStorage to all fetch headers.
- **Role Protection:** Admin routes are doubly protected. After going through the protect check, requests must pass the isAdmin validation, which ensures

that req.user.role === 'admin' or req.user.isAdmin === true before executing administrative controllers.

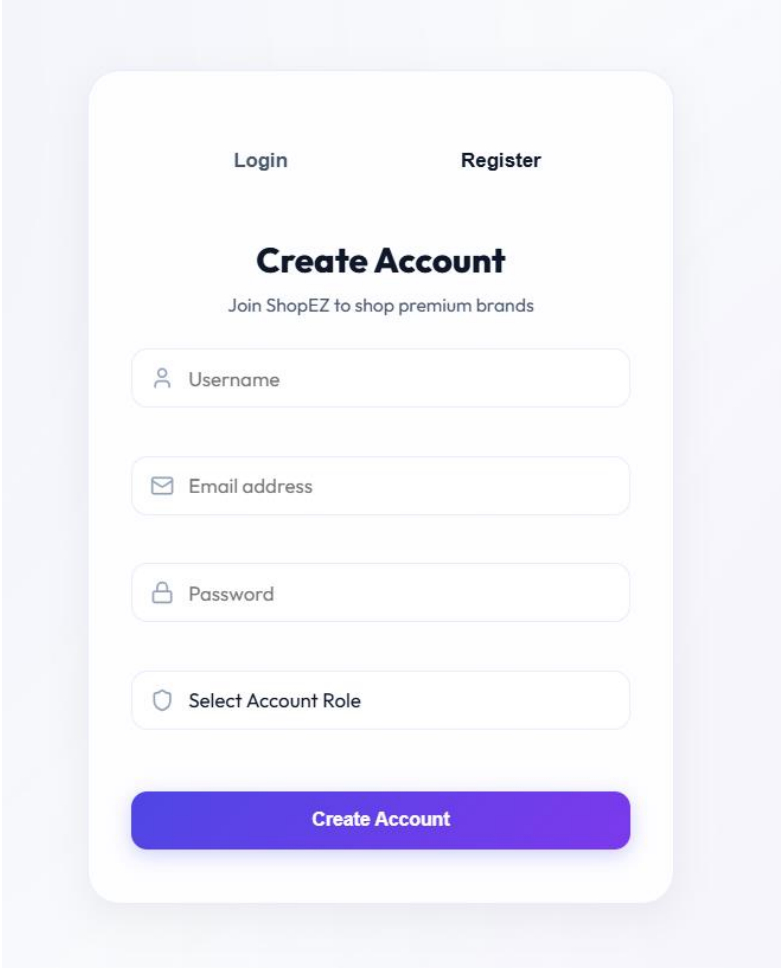
9. User Interface Design & Themes

Landing page

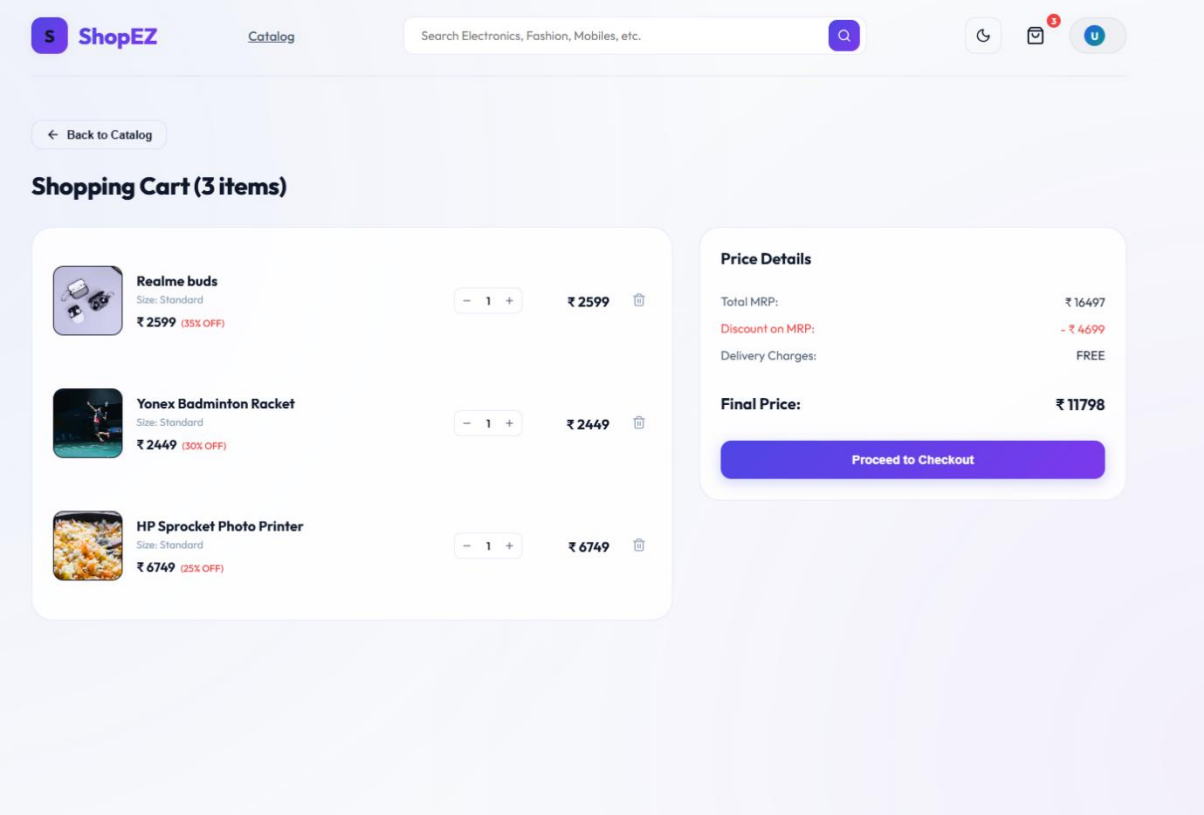




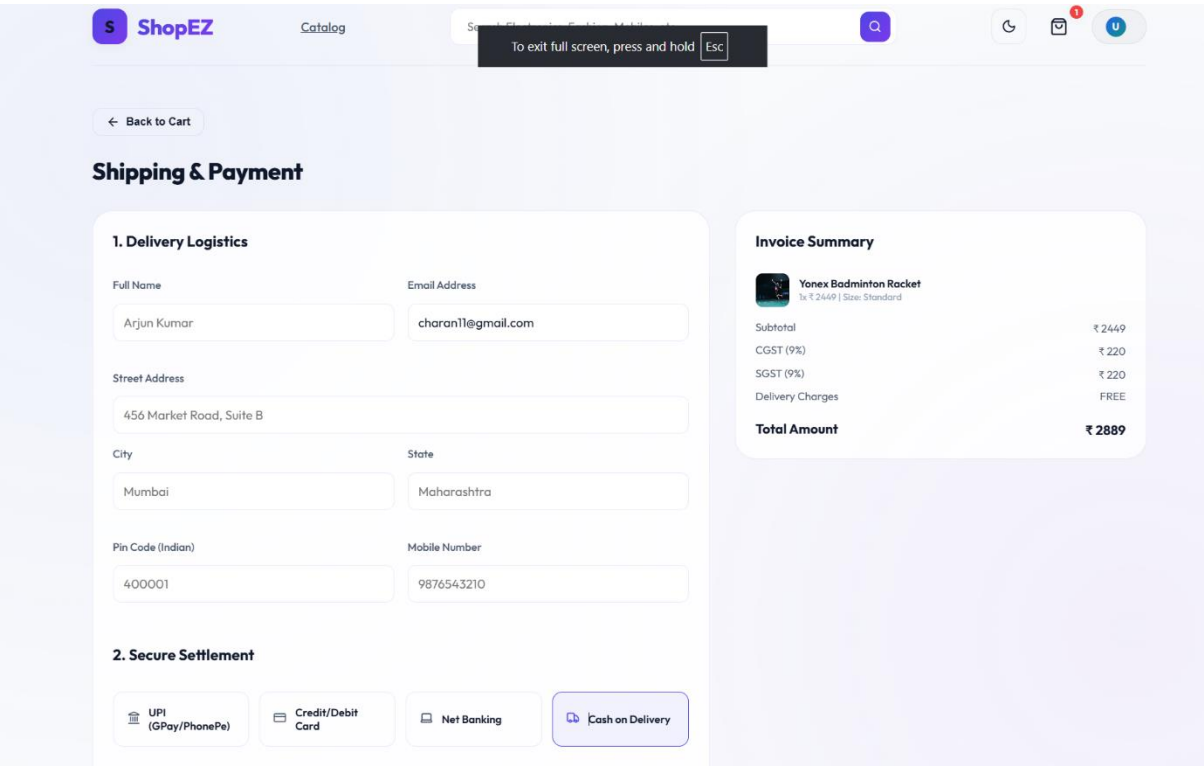
LOGIN PAGE


Login
Register
Create Account
Join ShopEZ to shop premium brands
Username
Email address
Password
Select Account Role
Create Account

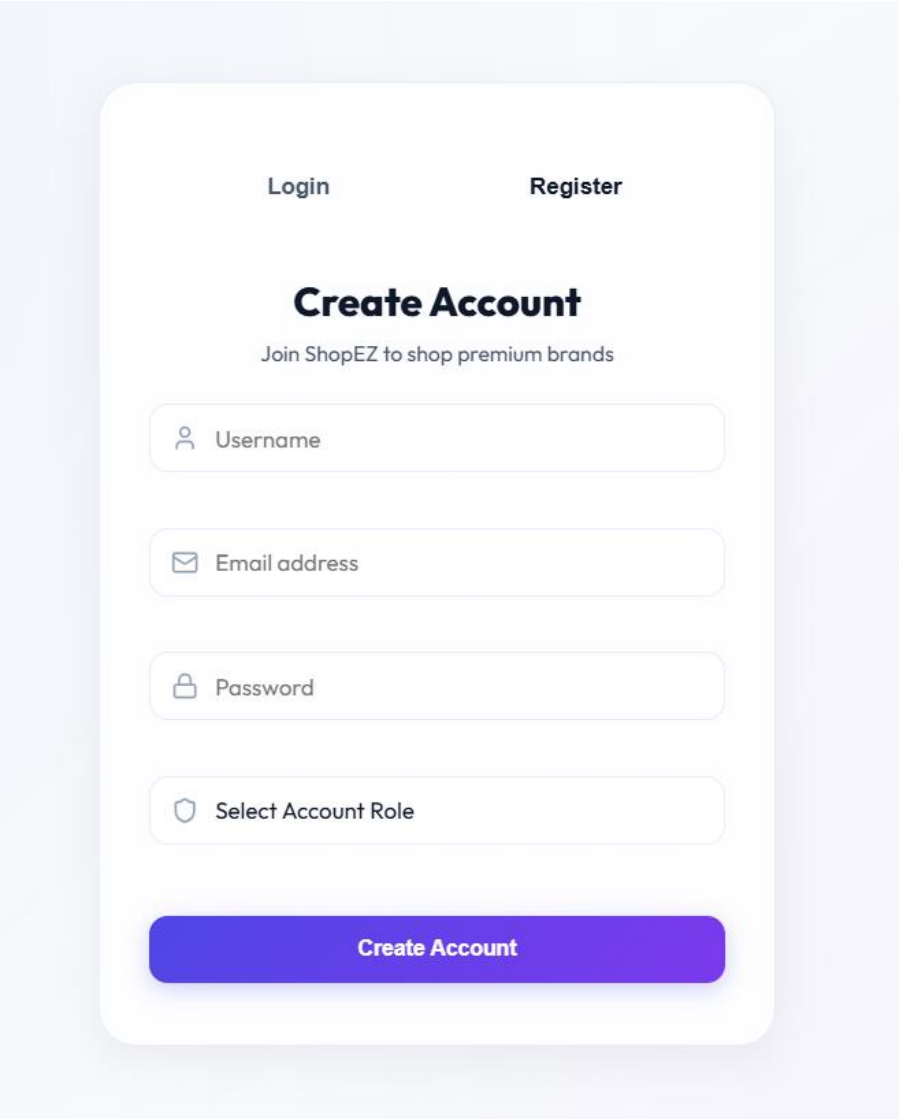
CART


ShopEZ
Catalog
Search Electronics, Fashion, Mobiles, etc.
Back to Catalog
Shopping Cart (3 items)
Realme buds
Size: Standard
₹ 2599 (35% OFF)
1
₹ 2599
Yonex Badminton Racket
Size: Standard
₹ 2449 (30% OFF)
1
₹ 2449
HP Sprocket Photo Printer
Size: Standard
₹ 6749 (25% OFF)
1
₹ 6749
Price Details
Total MRP:
₹ 16497
Discount on MRP:
- ₹ 4699
Delivery Charges:
FREE
Final Price:
₹ 11798
Proceed to Checkout

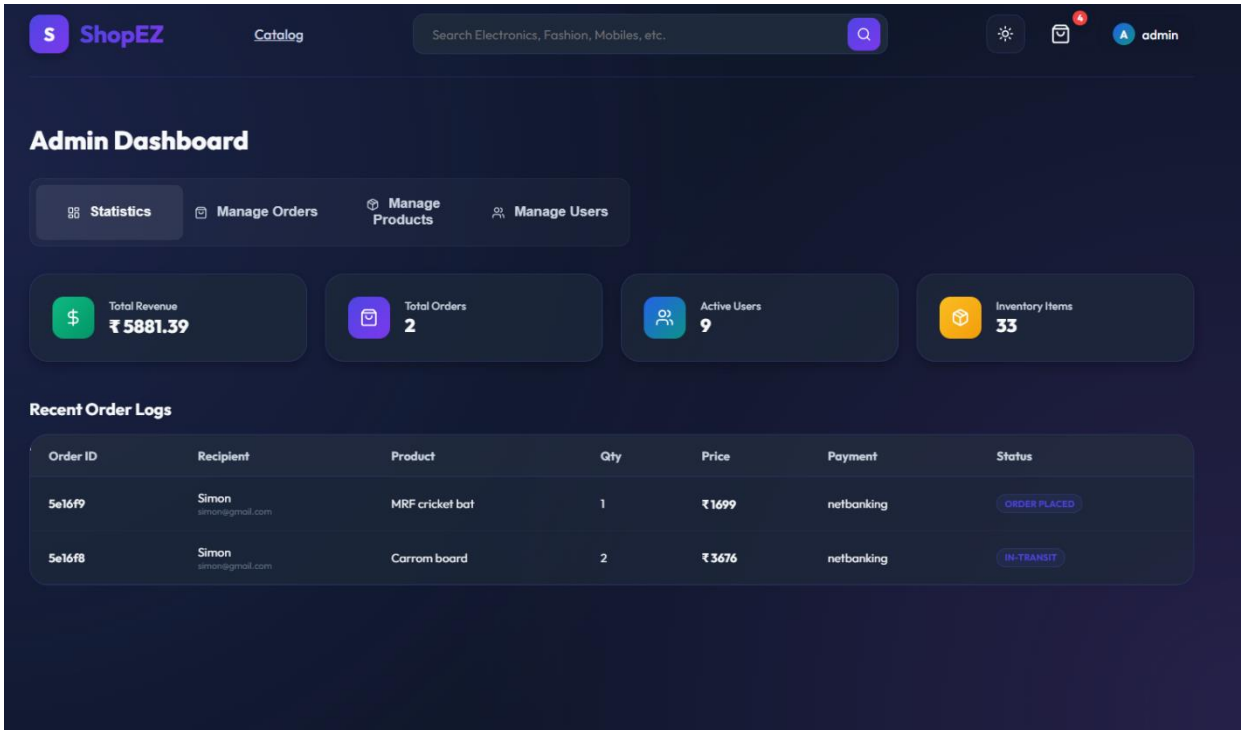
PAYMENT

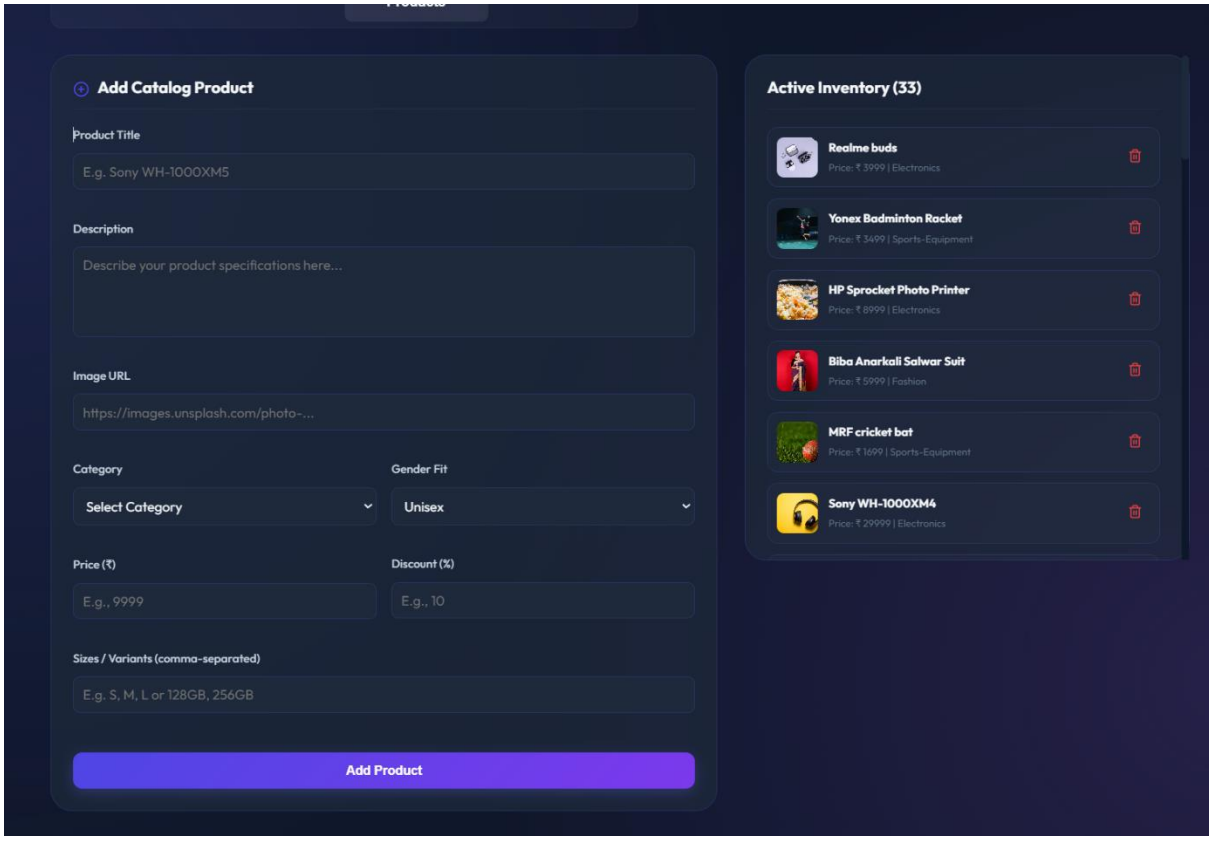

ShopEZ
Catalog
To exit full screen, press and hold Esc
Back to Cart
Shipping & Payment
1. Delivery Logistics
Full Name
Arjun Kumar
Email Address
charan11@gmail.com
Street Address
456 Market Road, Suite B
City
Mumbai
State
Maharashtra
Pin Code (Indian)
400001
Mobile Number
9876543210
2. Secure Settlement
UPI (GPay/PhonePe)
Credit/Debit Card
Net Banking
Cash on Delivery
Invoice Summary
Yonex Badminton Racket
1x ₹ 2449 | Size: Standard
Subtotal
₹ 2449
CGST (9%)
₹ 220
SGST (9%)
₹ 220
Delivery Charges
FREE
Total Amount
₹ 2889

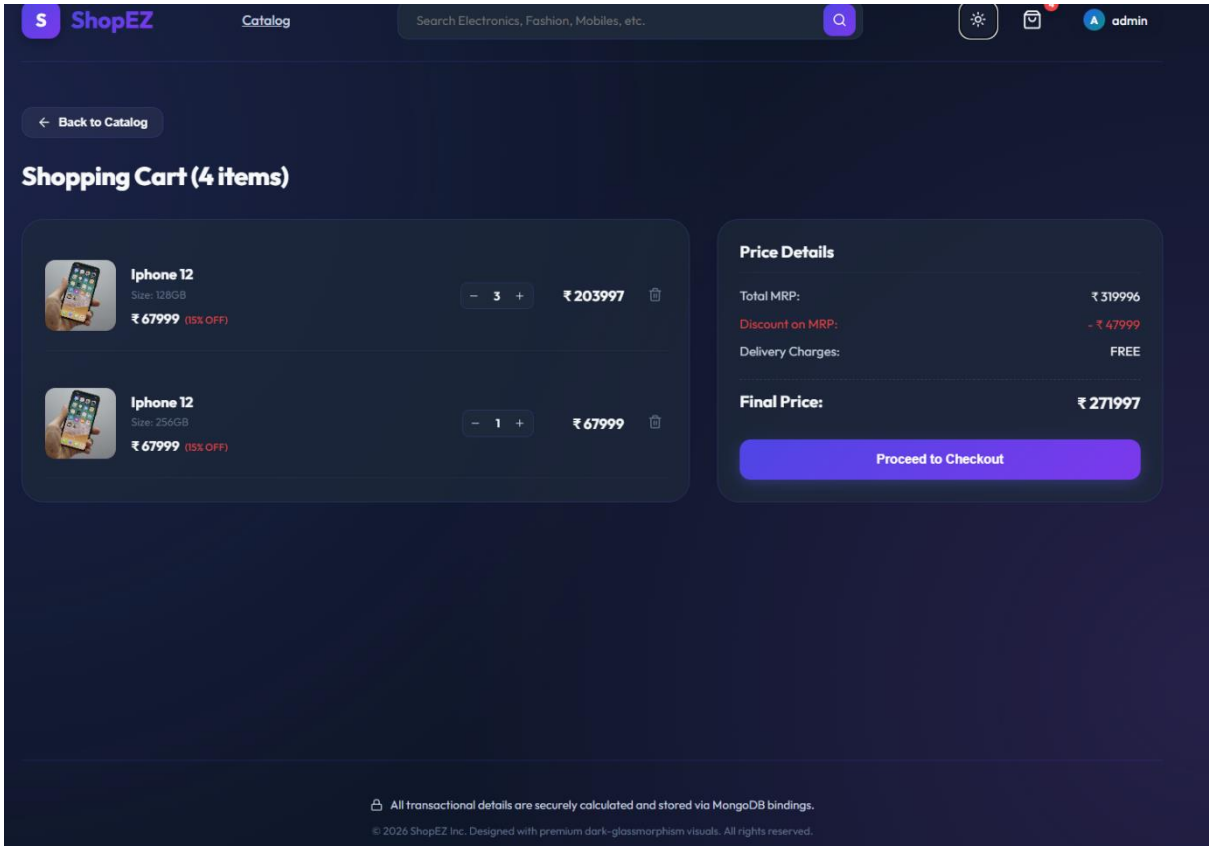
REGISTER PAGE



ADMIN PAGE


ShopEZ
Catalog
Search Electronics, Fashion, Mobiles, etc.
admin
Admin Dashboard
Statistics
Manage Orders
Manage Products
Manage Users
Total Revenue
₹ 5881.39
Total Orders
2
Active Users
9
Inventory Items
33
Recent Order Logs
Order ID
Recipient
Product
Qty
Price
Payment
Status
5e16f9
Simon
MRF cricket bat
1
₹1699
netbanking
ORDER PLACED
5e16f8
Simon
Carrom board
2
₹3676
netbanking
IN-TRANSIT


Add Catalog Product
Product Title
E.g. Sony WH-1000XM5
Description
Describe your product specifications here...
Image URL
https://images.unsplash.com/photo-...
Category
Select Category
Gender Fit
Unisex
Price (₹)
E.g., 9999
Discount (%)
E.g., 10
Sizes / Variants (comma-separated)
E.g. S, M, L or 128GB, 256GB
Add Product
Active Inventory (33)
Realme buds
Price: ₹ 3999 | Electronics
Yonex Badminton Racket
Price: ₹ 3499 | Sports-Equipment
HP Sprocket Photo Printer
Price: ₹ 8999 | Electronics
Biba Anarkali Salwar Suit
Price: ₹ 5999 | Fashion
MRF cricket bat
Price: ₹ 1699 | Sports-Equipment
Sony WH-1000XM4
Price: ₹ 29999 | Electronics



10. Testing

Currently, testing of API endpoints and UI widgets can be conducted through manual verification:

Manual Test Flow Checklist

1. **Authentication Test:** Register a customer account, log out, and log back in. Verify if credentials generate appropriate error messages.
2. **Catalog Navigation:** Apply various category, gender, and sorting configurations. Check search functionality.
3. **Shopping Cart:** Add products, update quantities, choose sizes, and check if calculations match expected outcomes.
4. **Checkout Procedure:** Populate the delivery fields, choose payment method, and complete order. Verify if the cart is cleared post-purchase.
5. **Profile Checks:** Navigate to the user profile. Ensure the order history matches recent checkouts. Cancel an order and verify the status update.

6. **Admin Verification:** Log in with an admin account. Access dashboard metrics, promotion switches, orders status logs, and create/edit catalog products.

Automated Integration Recommendations

- **Backend Testing:** Integrate mocha/jest combined with supertest to test controllers and route access protections.
- **Frontend Testing:** Configure vitest with @testing-library/react to test page rendering and Mock API interactions.

11. Known Issues & Limitations

1. **Admin Promote User Name Bug:** In adminController.js promotion confirmation string, user.name is output instead of user.username (schema definition uses username). This displays an undefined value in success responses.
2. **Stubbed Reviews Route:** The POST /api/products/:id/reviews endpoint is stubbed and does not save inputs. The product model currently lacks a dedicated reviews nested schema.
3. **Missing Client-side Input Validation:** The checkout form does not validate phone numbers, emails, or zip codes before sending them to the backend, relying on mock inputs.
4. **Batch Database Products Seeding:** Products query automatically calls the seeding method if counts drop below 30, which can slow down request responses during initial database sync.

12. Future Enhancements

- **Online Payment Gateway:** Integrate actual credit/debit card processors or gateways like Stripe or Razorpay.
- **Real-time Stock Control:** Subtract item amounts from product inventories upon checkout and block "Add to Cart" triggers when items go out of stock.
- **Review System Implementation:** Refactor the Mongoose Product schema to include reviews arrays containing user IDs, scores, and comment objects.
- **Automated Email Invoices:** Utilize pdfkit to generate PDF summaries and send email notifications to users upon purchase.

- **Search Engine Optimization (SEO):** Implement server-side rendering or React Helmet meta-tag controllers to inject metadata on dynamic catalog routes.